

sAIfer Lab

Joint lab on Safety and Security of AI

# Elementi di statistica

# Statistica

La **statistica** fornisce metodi matematici finalizzati alla conoscenza quantitativa e qualitativa dei fenomeni collettivi mediante la raccolta, la sintesi e l'analisi dei dati

Statistica **descrittiva**: descrizione di fenomeni attraverso l'organizzazione, la sintesi e la presentazione dei dati nella forma di indici e grafici

Statistica **inferenziale**: permette di *generalizzare* i risultati ottenuti su un campione a una popolazione più ampia, mediante metodi probabilistici

# Popolazione e campione

**Popolazione**: l'insieme di tutte le entità (individui, oggetti, prezzi, ecc.), dette *unità statistiche*, delle quali si vogliono indagare determinate caratteristiche

- esempio: la diffusione di una certa malattia tra *tutti* gli abitanti di una certa area geografica

Collezionare e analizzare dati da un'intera popolazione è spesso difficile e costoso: le conclusioni vengono tratte da analisi condotte su un suo sottoinsieme rappresentativo

**Campione**: sottoinsieme di unità statistiche *rappresentativo* dell'intera popolazione, dal quale trarre, con margini di errore contenuti, indicazioni sulle caratteristiche della totalità

# Statistica: le variabili

Una **variabile** è una grandezza o un elemento che può essere osservato o misurato. Esempi:

- un individuo può avere o meno una certa malattia

Variabile **dipendente**: è una grandezza il cui valore è determinato da una o più altre grandezze. Esempio:

- frazione di individui di una popolazione o di un suo campione affetti da una data malattia

Variabile **indipendente**: è una grandezza che può essere modificata arbitrariamente, e che influisce sulle variaibli dipendenti. Esempio:

- somministrazione di un farmaco contro una certa malattia a un campione di individui (può modificare la percentuale di individui affetti da tale malattia)

# Statistica: tipi di dato

Dati **qualitativi** (o **categorici**)

- **nominali**: insieme di categorie senza un ordinamento logico (es: sesso, presenza di una malattia)
- **ordinali**: insieme di categorie tra le quali esiste un ordinamento (es.: titolo di studio)

Dati **quantitativi** (o **numerici**)

- **discreti**: assumono valori numerici discreti, in numero finito o numerabile (es.: numero di figli)
- **continui**: assumono valori reali (es.: temperatura, altezza)
- **di intervallo**: valori numerici con differenza costante e zero *arbitrario* (es.: temperatura)
- **di rapporto**: valori numerici con differenza costante e zero *non arbitrario* (es.: altezza)

# Statistica descrittiva

Di seguito si presentano gli elementi essenziali della statistica **descrittiva** relativi alla sintesi dei dati mediante indici e alla loro presentazione mediante grafici, e le librerie Python per il calcolo di indici e il tracciamento di grafici

**Indici numerici**

- misure di tendenza centrale: media, mediana, moda
- misure di dispersione (o di variabilità): varianza, deviazione standard, quantili (quartili e percentili)
- covarianza, coefficiente di correlazione

**Grafici**

- grafici X-Y
- istogrammi

# Librerie Python per la statistica descrittiva

Librerie presentate in questo corso

- **statistics** (libreria *built-in* per il calcolo di indici statistici)
- **Matplotlib** (libreria per il tracciamento di grafici)
- **Pandas** (verrà presentata più avanti)

Altre librerie

- **NumPy**
- **SciPy**

```
import statistics
import matplotlib
import pandas
import numpy
import scipy.stats
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Misure di tendenza centrale: media, mediana, moda

# Media

La **media** (aritmetica) è la misura più comune per riassumere dati quantitativi (numerici)

La media di un campione di $N$ valori $\{x_1, x_2, \dots, x_N\}$ è definita come:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$$

# Calcolo della media in Python

Un semplice programma per valori memorizzati in una lista:

```
valori = [181.1, 170, 172.5, 174, 168]

somma = 0
for valore in valori:
    somma = somma + valore
media = somma/len(valori)
print(media)
```

Si può anche usare la funzione `sum`:

```
valori = [181.1, 170, 172.5, 174, 168]

media = sum(valori)/len(valori)
print(media)
```

# La media con la libreria `statistics`

Si calcola mediante la funzione `mean()`

```
import statistics

valori = [181.1, 170, 172.5, 174, 168]

media = statistics.mean(valori)

print(media)
```

# Media pesata

A ogni osservazione $x_i$ è associato un peso numerico $w_i$ che ne definisce il contributo nel calcolo della media:

$$\{(x_1, w_1), (x_2, w_2), \dots, (x_N, w_N)\}$$

La media pesata è definita come:

$$\bar{x} = \frac{1}{W} \sum_{i=1}^{N} w_i \cdot x_i, \quad \text{dove} \quad W = \sum_{i}^{N} w_i$$

Esempio: media dei voti degli esami pesata in base al numero di crediti, $\{(27, 6), (25, 5), (30, 8), (28, 6), (24, 8)\}$:

$$\frac{27 \cdot 6 + 25 \cdot 5 + 30 \cdot 8 + 28 \cdot 6 + 24 \cdot 8}{6 + 5 + 8 + 6 + 8} = \frac{887}{33} \cong 26.88$$

# Calcolo della media pesata in Python

Osservazioni e pesi corrispondenti memorizzati in due liste

```
import math

voti = [27, 25, 30, 28, 24]
crediti = [6, 5, 8, 6, 8]

numeratore = 0
for i in range(len(voti)):
    numeratore = numeratore + voti[i]*crediti[i]

media_pesata = numeratore/sum(crediti)
print(media_pesata)
```

# La media pesata con la libreria `statistics`

Si calcola mediante la funzione `fmean()`

```
import statistics

voti = [27, 25, 30, 28, 24]
crediti = [6, 5, 8, 6, 8]

media_pesata = fmean(voti, crediti)

print(media_pesata)
```

# Mediana

La **mediana** di un campione di dati qualitativi (solo se di tipo *ordinale*) o numerici è il valore rispetto al quale una metà dei restanti valori risulta superiore e l'altra metà inferiore

Esempio: un campione di $N = 5$ valori

$$\{174, 172.5, 170, 181.1, 168\}$$

La mediana è $172.5$:

- due degli altri quattro valori sono inferiori ($168$ e $170$)
- due sono superiori ($174$ e $181.1$)

# Calcolo della mediana

La mediana corrisponde al valore centrale della sequenza di valori ordinata in senso crescente (o decrescente). Esempio:

$$\{174, 172.5, 170, 181.1, 168\} \rightarrow \{168, 170, \mathbf{172.5}, 174, 181.1\}$$

Se $N$ è pari, la mediana è definita come la media dei due valori centrali della sequenza ordinata. Esempio ($N = 6$):

$$\{174, 172.5, 177, 170, 181.1, 168\} \rightarrow \{168, 170, \mathbf{172.5}, \mathbf{174}, 177, 181.1\}$$

Mediana: $(172.5 + 174)/2 = 173.25$

# Mediana vs media

La media è sensibile alla presenza di **anomalie** nei dati, ovvero valori molto maggiori o molto minori di tutti gli altri (*outlier*). Esempio:

- $\{174, 172.5, 170, 181.1, 168\}$: la media è $173.12$
- $\{174, 172.5, 170, 520, 168\}$: la media è $240.9$

La mediana è invece un indice più **robusto** agli *outlier*

Per esempio, in entrambi i casi precedenti il valore della mediana è pari a $172.5$: non è quindi influenzato dalla presenza di un'anomalia nel secondo campione

# Calcolo della mediana in Python

Si può usare la funzione `sorted()`: data una lista, restituisce una nuova lista con gli stessi elementi ordinati in senso crescente

- se il numero $N$ di valori è **dispari**, la mediana è il valore in posizione centrale, il cui indice è `int(N/2)`
- se $N$ è pari, la mediana è la media dei due valori in posizione centrale, i cui indici nella lista dei valori sono `int(N/2) - 1` e `int(N/2)`
  (si usa `int()` anche se $N$ è pari, perché l'operatore `/` produce un valore di tipo `float`, mentre l'operatore di indicizzazione accetta solo valori di tipo `int`)

# Calcolo della mediana in Python

```
valori = [181.1, 170, 172.5, 174, 168]
# provare anche: [168, 177, 170, 172.5, 174, 181.1]

n = len(valori)
ordinati = sorted(valori) # [168, 170, 172.5, 174, 181.1]

i = int(n/2)
if n % 2 == 1: # numero dispari di osservazioni
    mediana = ordinati[i]
else:          # numero pari di osservazioni
    mediana = (ordinati[i - 1] + ordinati[i])/2

print(mediana)
```

# La mediana con la libreria `statistics`

Con statistics

```
import statistics

valori = [181.1, 170, 172.5, 174, 168]
# provare anche: [168, 177, 170, 172.5, 174, 181.1]

mediana = statistics.median(valori)
print(mediana)
```

# La mediana di dati categorici

La definizione di mediana vale anche per dati **qualitativi**, purché di tipo **ordinale**, ossia tali che tra i loro valori sussista un ordinamento

Nel caso caso di un numero pari di osservazioni, non essendo possibile calcolare la media aritmetica di valori non numerici, le mediane sono due: mediana inferiore (*lower median*) e mediana superiore (*upper median*)

# Calcolo della mediana di dati categorici in Python

La funzione `sorted()` può ordinare anche stringhe

I valori categorici non sono supportati da `statistics`

```
# titoli di studio: a1 medie; a2 superiori; a3 università
titoli = ['a1', 'a2', 'a3', 'a1', 'a1', 'a2', 'a3', 'a2',
          'a3', 'a2', 'a3', 'a3', 'a3', 'a2', 'a3']

n = len(titoli)
ordinati = sorted(titoli)

i = int(n/2)
if n % 2 == 1: # numero dispari di osservazioni
    mediana = ordinati[i]
    print(mediana)
else:          # numero pari di osservazioni
    mediana_inf = ordinati[i - 1])
    mediana_sup = ordinati[i])
    print(mediana_inf, mediana_sup)
```

# Moda

La **moda** è il valore che si presenta con maggiore frequenza in un insieme di dati

È definita sia per dati quantitativi che qualitativi, anche di tipo nominale, ovvero non ordinabili

Se la frequenza maggiore è comune a più valori la moda è costituita dall'insieme di tali valori, e i dati sono detti *multimodali*

Per calcolare la moda è necessario contare il numero di occorrenze di ciascun valore in un dato campione, ovvero quante volte ciascun valore compare nel campione

Nel caso di dati quantitativi anche la moda, come la mediana, non è influenzata dalla presenza di anomalie

# Moda

Esempio: numero di figli per coppia in un campione di $N = 10$ famiglie: $\{1, 2, 1, 0, 1, 0, 2, 3, 0, 1\}$

- 0: 3 occorrenze
- 1: 4 occorrenze
- 2: 2 occorrenze
- 3: 1 occorrenza

La moda è quindi 1

# Calcolo della moda in Python

La scrittura di un programma per il calcolo della moda è lasciato come esercizio

# La moda con la libreria `statistics`

Due funzioni

- `mode()`, nel caso in cui sia noto a priori che nei dati è presente un'unica moda
- `multimode()`, nel caso in cui il campione possa essere multimodale (la funzione restituisce una lista con tutti i valori che costituiscono la moda, al limite uno solo)

```
import statistics

x = [174, 170, 172.5, 170, 174, 168.0]
moda = statistics.multimode(x)
# moda = statistics.mode(x), se si è sicuri che
#  la moda sia unica
print(moda)
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Misure di dispersione (o di variabilità)

# Misure di variabilità

Le misure di tendenza centrale non sono sufficienti per descrivere dati **quantitativi**: sono necessarie misure di **variabilità** che consentano di quantificare la **dispersione** rispetto alla misura di tendenza centrale

- varianza
- deviazione standard
- quantili

# La varianza

La **varianza** è una misura della dispersione di un insieme di valori rispetto alla loro **media**. È definita come lo scarto quadratico medio rispetto alla loro media aritmetica

Valori maggiori della varianza indicano una maggiore dispersione dei dati rispetto alla loro media, mentre valori più piccoli indicano una maggiore concentrazione intorno alla media

# Varianza della popolazione e di un campione

- Per una **popolazione** di $n$ individui:

$$\sigma^2 = \frac{1}{n}\sum_i^n (x_i - \mu)^2$$

dove $\mu$ è la media della popolazione

- Per un campione di $N$ individui:

$$s^2 = \frac{1}{n-1}\sum_i^n (x_i - \bar{x})^2$$

dove $\bar{x}$ è la **media campionaria**

NOTA: si divide per $n-1$, non per $n$ (Bessel's correction), per ottenere una stima non polarizzata di $\sigma^2$

# Calcolo della varianza in Python

```
x = [174, 170, 172.5, 170, 174, 168]

n = len(x)

media = sum(x)/n

somma = 0
for elemento in x:
    somma = somma + (elemento - media)**2

varianza = somma/(n - 1)
print(varianza)
```

# La varianza con `statistics`

Varianza della popolazione: si usa la funzione `pvariance()`

Varianza di un campione: si usa la funzione `variance()`

```
import statistics

x = [174, 170, 172.5, 170, 174, 168]

varianza = statistics.variance(x)

print(varianza)
```

# Esempio

Una grandezza quantitativa nell'intervallo $[0,1)$ misurata su una **popolazione** di $n$ individui può essere simulata generando $n$ numeri casuali in tale intervallo mediate la funzione `random()` della libreria omonima (si veda la slide seguente). Si possono quindi calcolare la media e la varianza della popolazione

Successivamente, estraiamo casualmente un campione (sottoinsieme) di $N$ valori dalla popolazione, con $N \ll n$, mediante la funzione `sample()` della stessa libreria (si veda la slide seguente), e calcoliamo la media e la varianza **campionarie**, che costituiranno una stima dei corrispondenti indici della popolazione

Al crescere di $n$ si osserverà che le stime campionarie si avvicinano ai valori della popolazione

# Esempio

La funzione `random()` genera un numero casuale in virgola mobile (di tipo `float`) compreso nell'intervallo $[0,1)$

La funzione `sample()` estrae un sottoinsieme casuale di un dato numero di valori da una sequenza (per esempio, una lista)

# Esempio

```
import statistics
import random

n = 10000  # dimensione della popolaziome
list_N = [5, 10, 100, 1000] # dimensione dei campioni

valori = []
for i in range(n):
    valori = valori + [random.random()]

media = statistics.mean(valori)
varianza = statistics.pvariance(valori)

print(f'media: {media:0.5f}')
print(f'varianza: {varianza:0.5f}')
```

(cont.)

# Esempio

```
for N in list_N:
    campioni = random.sample(valori, N)
    media_campionaria = statistics.mean(campioni)
    varianza_campionaria = statistics.variance(campioni)
    print(f'dimensione del campione: {N}')
    print(f' media campionaria: {media_campionaria:0.5f}')
    print(f' varianza campionaria:
                    {varianza_campionaria:0.5f}')
```

# La deviazione standard

La **deviazione standard**, o **scarto quadratico medio**, è una misura della dispersione rispetto alla media di un insieme di osservazioni, ed è definita come la **radice quadrata** della **varianza**

Nel caso di una popolazione si indica con $\sigma$, nel caso di un campione si indica con $s$

# La deviazione standard con `statistics`

Per la popolazione si usa la funzione `pstdev()`

Per un campione si usa la funzione `stdev()`

```
import statistics

x = [174, 170, 172.5, 170, 174, 168]

std_dev = statistics.stdev(x)

print(std_dev)
```

# Quantili, quartili e percentili

Un **quantile** di ordine $\alpha \in [0,1]$ è un valore $q_\alpha$ tale che una frazione pari a $\alpha$ di una popolazione o un campione ha valori minori o uguali a $q_\alpha$

I **quartili** sono quantili corrispondenti a $\alpha = 0.25, 0.5, 0.75$

- primo quartile ($q_{0.25}$): il 25% dei valori è inferiore a $q_{0.25}$
- secondo quartile ($q_{0.5}$): il 50% dei valori è inferiore a $q_{0.5}$ ($q_{0.5}$ corrisponde quindi alla **mediana**)
- terzo quartile ($q_{0.75}$): il 75% dei valori è inferiore a $q_{0.75}$

Analogamente, i **percentili** sono quantili corrispondenti a $\alpha = 0.01, 0.02, 0.03, \dots, 0.98, 0.99$

# Quantili, quartili e percentili

Notare che quartili e percentili suddividono (in lilnea di principio) un insieme di valori in sottoinsiemi contenenti lo **stesso** numero di **valori**

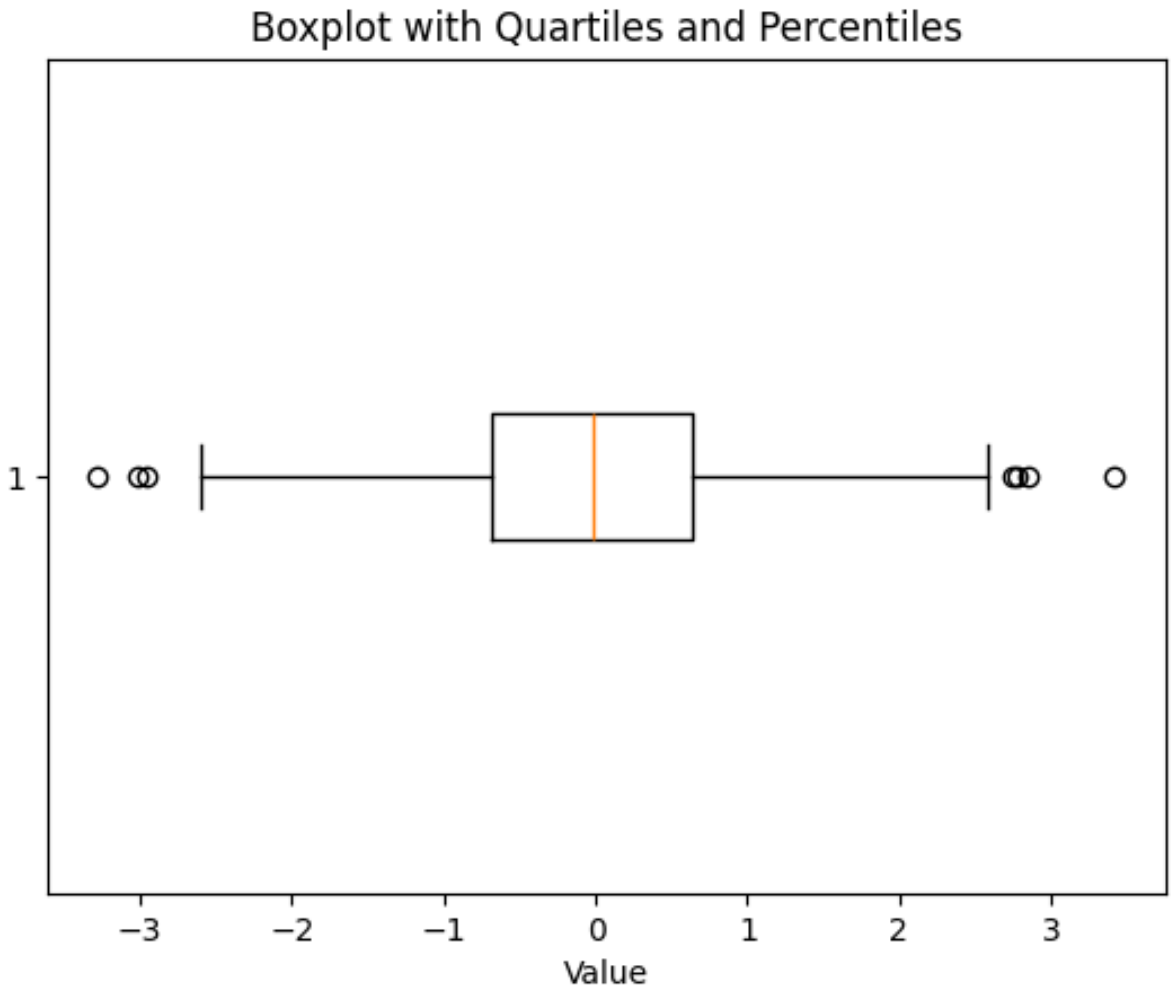
**Esempio**: il numero di elementi di un campione che si trovano tra i quartili $q_{0.25}$ e $q_{0.5}$ è identico al numero di elementi tra i quartili $q_{0.2}$ e $q_{0.75}$

# Esempio: *box plot*

Il *box plot* o *box-and-whisker plot* (si veda la figura seguente) è una comune rappresentazione di un insieme di valori in termini di quantili

- linea centrale (rossa): mediana (50° percentile)
- bordi della "scatola" (*box*): primo quartile (25° percentile) e terzo quartile (75° percentile)
- segmenti verticali (*whiskers*): intervallo dei dati, escludendo i valori considerati anomali (rappresentati dai cerchi)

# Esempio: *box plot*



# Esempio

Consideriamo 10 valori ordinati:

`[10, 30, 40, 50, 60, 70, 71, 72, 73, 74]`

Calcoliamo il percentile $q_{0.60}$:

**Posizione** del percentile: $60/100 \cdot (n+1) = 6.6$, dove $n = 10$ è il numero di valori. Il percentile si trova quindi tra il $6^{\circ}$ valore (70) e il $7^{\circ}$ valore (71)

Il **valore** del percentile si calcola mediante interpolazione:

$$q_{0.60} = 70 + (71 - 70) \times 0.6 = 70.6$$

(si usa $y = ax + b$, dove $x$ è la posizione del percentile e $y$ il suo valore)

# Esempio

Calcoliamo ora il terzo quartile, $q_{0.75}$

**Posizione** del quartile: $3/4 \cdot (n+1) = 8.25$. Il quartile si trova tra l'$8^{\circ}$ valore (72) e il $9^{\circ}$ (73)

Il **valore** esatto del quartile si trova, come in precedenza, mediante interpolazione:

$$q_{0.75} = 72 + (73 - 72) \times 0.25 = 72.25$$

# I quantili con `statistics`

Si usa la funzione `quantiles()`, specificando come secondo argomento il numero di quantili, mediante la sintassi `n = ...`

```
import statistics

valori = [174, 170, 172.5, 170, 174, 168, 151.2, 177, 179.3]

print('Quartili q_25, q_50, q_75:\n')
print(statistics.quantiles(valori, n = 4))
# [169.0, 172.5, 175.5]
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Coefficiente di correlazione

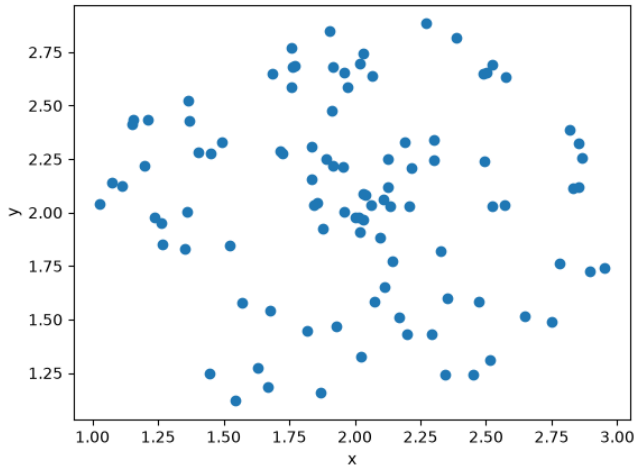
# Dipendenza lineare tra due grandezze

Spesso si ha la necessità di studiare la relazione tra due variabili $x$ e $y$, osservando un campione costituito da un insieme di coppie di valori $\{(x_1, y_1), \dots, (x_N, y_N)\}$

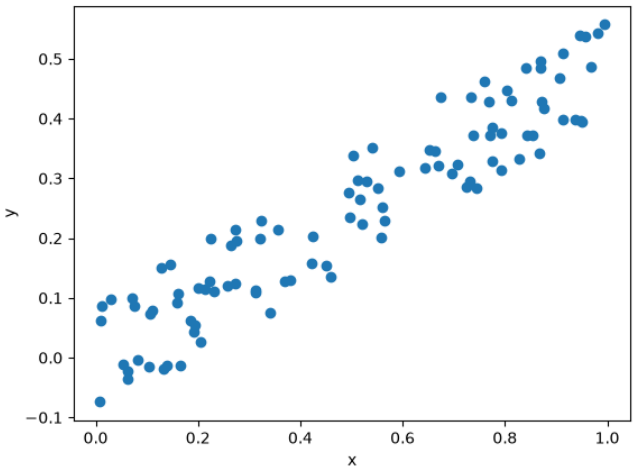
Esempio: altezza e peso di una popolazione di individui

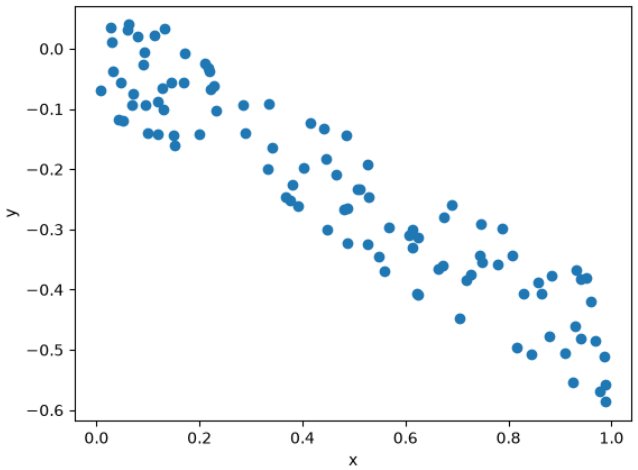
Il tipo più semplice di dipendenza è quella **lineare**: in questo caso si tratta di valutare in quale misura tra le coppie $(x_i, y_i)$ esista una relazione del tipo $y = ax + b$

# Esempio

a) 


a) assenza di dipendenza lineare
b) forte dipendenza lineare (diretta)
c) forte dipendenza lineare (inversa)

b) 


c) 

# Coefficiente di correlazione

Dato un campione composto da coppie di valori $\{(x_1, y_1), \ldots, (x_N, y_N)\}$, il **coefficiente di correlazione** è definito come:

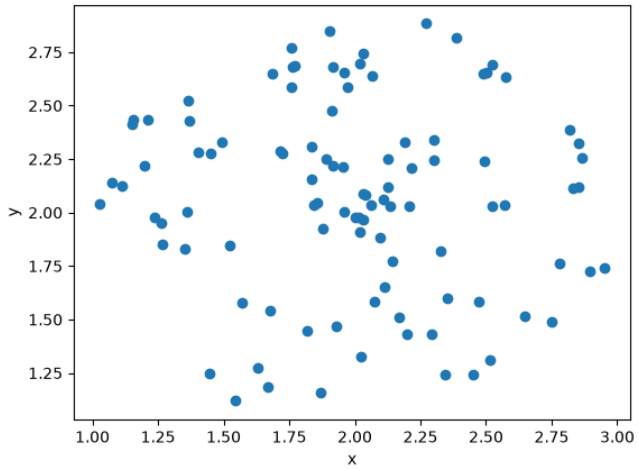
$$\rho = \frac{\sum_{i=1}^{N}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{N}(\mathrm{y}_i - \overline{y})^2}}$$

I suoi valori appartengono all'intervallo $[-1, +1]$, e misurano il grado di **dipendenza lineare** tra $x$ e $y$

- valori vicini a +1 e a -1 indicano una relazione lineare forte (diretta e inversa, rispettivamente)
- valori prossimi a 0 indicano l'assenza di relazione lineare

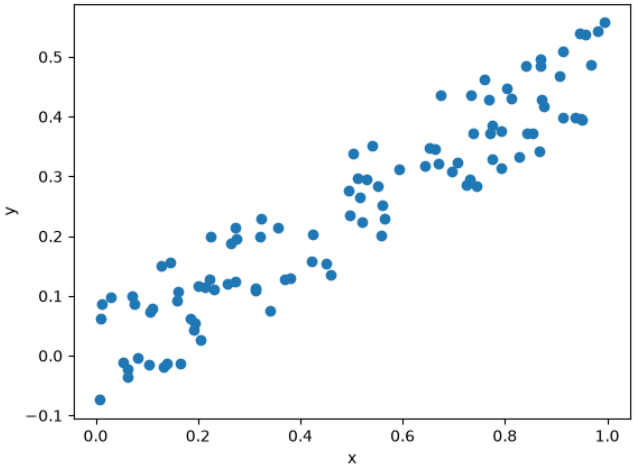
# Esempio

a)

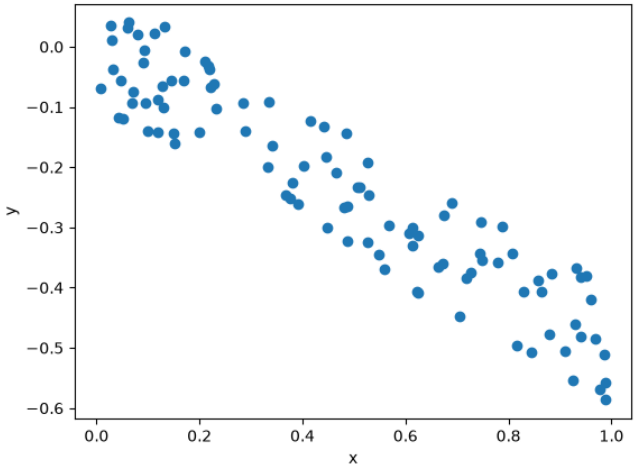



a) $\rho = -0.06$
b) $\rho = +0.94$
c) $\rho = -0.93$

b)




c)



# Il coefficiente di correlazione con `statistics`

Si usa la funzione `correlation()`

```
import statistics

x = [110, 112, 117, 105, 141, 140, 133, 128, 139, 131]
y = [6, 7, 7, 5, 12, 11, 9, 9, 13, 10]


rho = statistics.correlation(x, y)
print(f'Coefficiente di correlazione: {rho:0.2f}')  # 0.96
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Visualizzazione dei dati

# Visualizzazione dei dati

Al pari degli indici numerici, la visualizzazione può (e deve) essere usata per la descrizione dei dati

Principali tipi di grafici

- grafici X-Y
- diagrammi a dispersione (*scatter plot*)
- istogrammi
- grafici a torta (*pie chart*) e a barre (*bar chart*)

# Libreria `Matplotlib`: la funzione plot

`Matplotlib` è un'utile libreria Python per la generazione di grafici

Le funzioni di interesse per questo corso fanno parte del modulo `pyplot` di Matplotlib

Per approfondimenti:

matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

# Grafici X-Y: la funzione `plot`

La funzione `plot()` genera un grafico dei valori di $y$ in funzione di $x$: i suoi argomenti sono due liste di numeri della stessa lunghezza, contenenti rispettivamente i valori di $x$ e quelli di $y$

Esempio: un insieme di temperature misurate in giorni diversi (giorno 1: 5°, giorno 2: 7°, giorno 5: 9°, e così via)

```
temperature = [5, 7, 9, 11, 8, 11, 10, 15]
giorni = [1, 2, 5, 10, 18, 19, 20, 30]
```

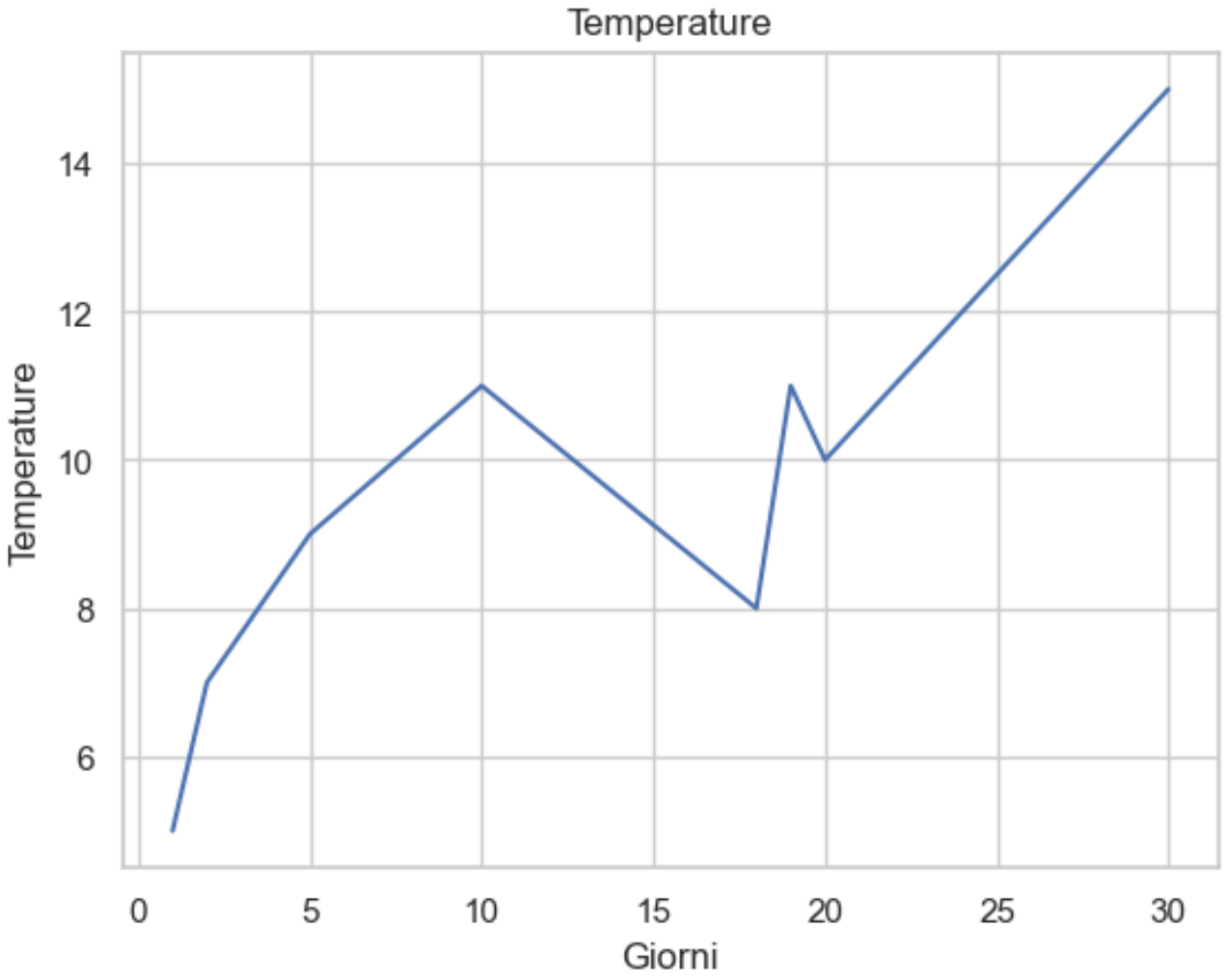
Si vuole ora tracciare un grafico delle coppie $(x, y)$ dove i valori di $x$ rappresentano i giorni e i valori di $y$ rappresentano le temperature

# Grafici X-Y: la funzione plot

```
import matplotlib.pyplot as plt

plt.plot(giorni, temperature)
# provate come terzo parametro di plot:
#  '-.', '.', 'o', 'r'
plt.title("Temperature")
plt.xlabel('Giorni')
plt.ylabel('Temperature')
plt.show()
```

# Grafici X-Y: la funzione `plot`



# Grafici X-Y: la funzione `plot`

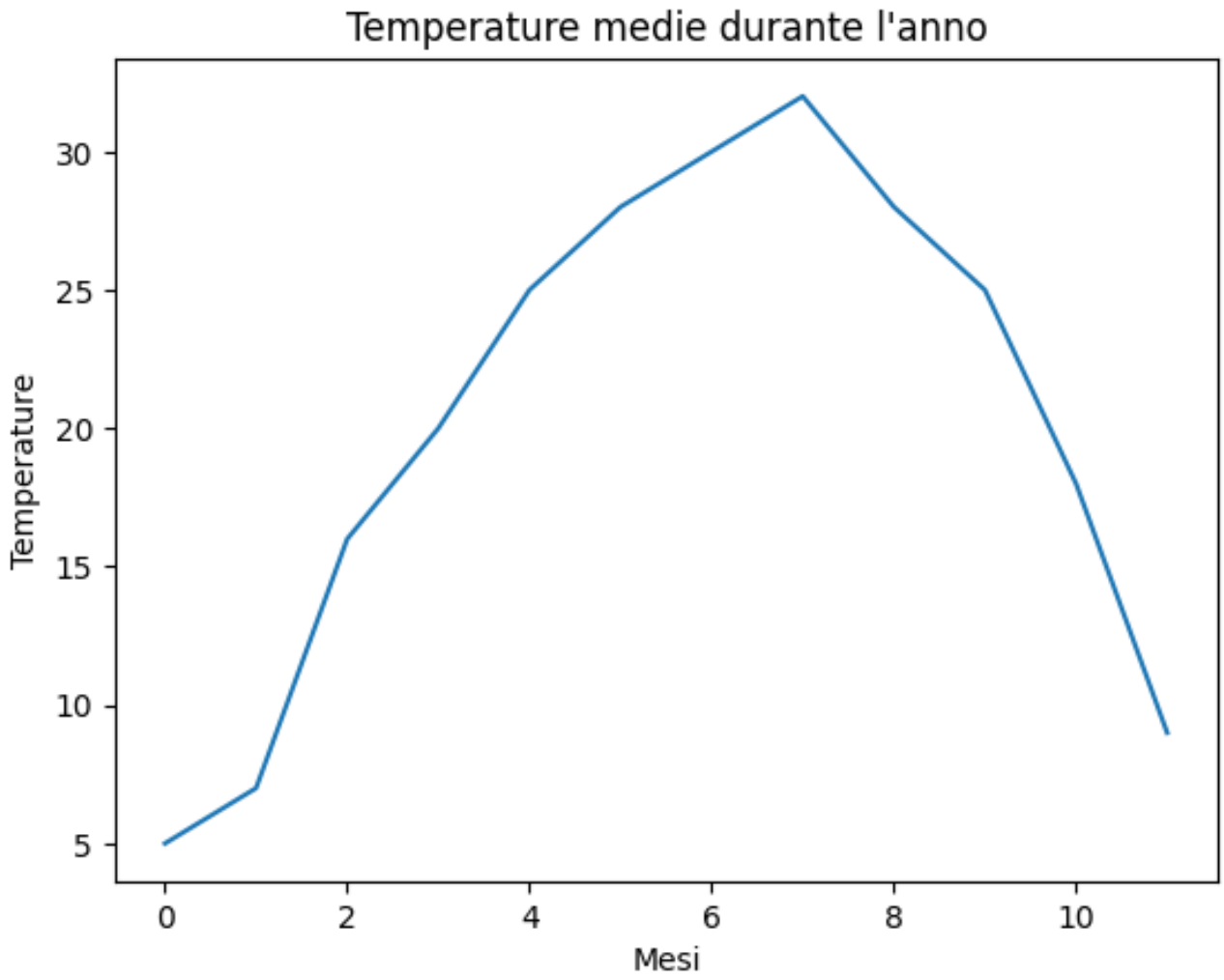
La funzione `plot()` può anche essere usata per rappresentare i valori di $y$ rispetto alla sequenza predefinita di valori di $x$ pari a 0, 1, 2, … (sempre in numero identico a quelli di $y$)

```
import matplotlib.pyplot as plt

temperature = [5, 7, 16, 20, 25, 28, 30, 32, 28, 25, 18, 9]

plt.plot(temperature)
plt.title("Temperature medie durante l'anno")
plt.xlabel('Mesi')
plt.ylabel('Temperature')
plt.show()
```

# Matplotlib: plot con i soli valori y



# Diagrammi a dispersione: la funzione `scatter`

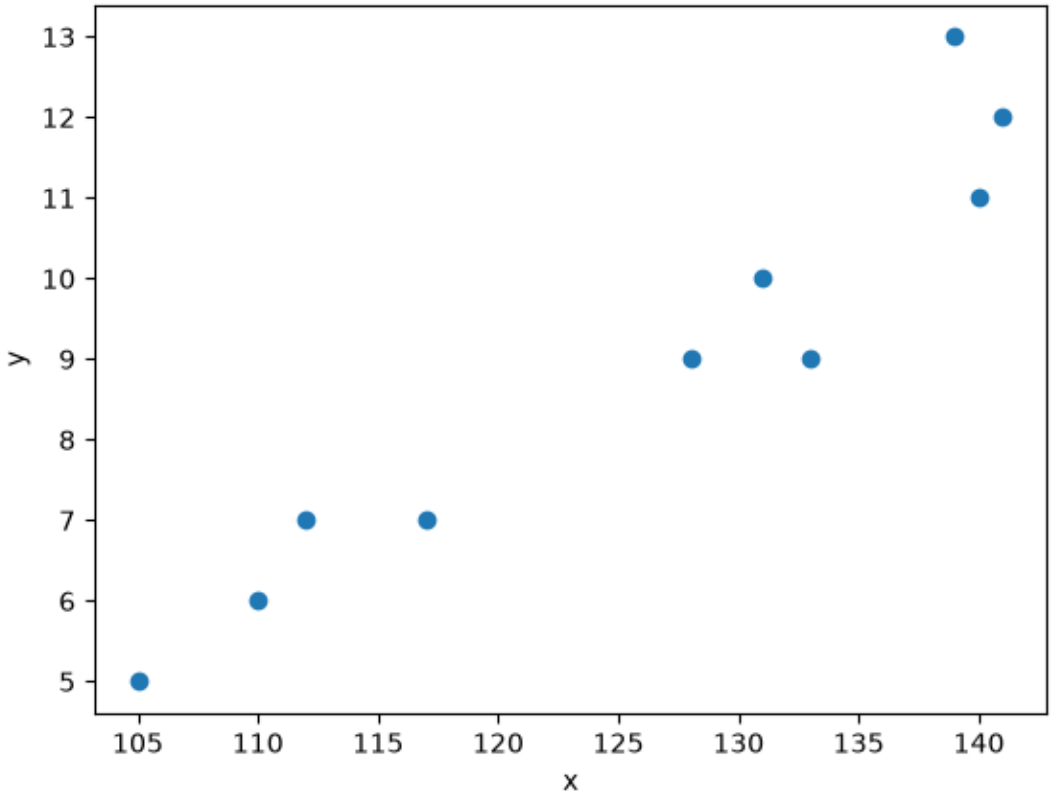
Per tracciare un insieme di punti $\{(x_1, y_1), \dots, (x_N, y_N)\}$ non connessi da una linea si può usare la funzione `scatter()`

```
import matplotlib.pyplot as plt

x = [110, 112, 117, 105, 141, 140, 133, 128, 139, 131]
y = [6, 7, 7, 5, 12, 11, 9, 9, 13, 10]

plt.scatter(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

# Diagrammi a dispersione: la funzione `scatter`



# Istogrammi

it.wikipedia.org/wiki/Istogramma

Gli **istogrammi** sono solitamente creati a partire da *dati numerici*

L'istogramma viene costruito suddividendo i dati in **intervalli** (*bin*) e calcolando il numero di valori che rientrano in ciascun intervallo, oppure la loro frequenza

L'area della superficie di ogni rettangolo coincide con il numero di valori o la frequenza associata alla classe cui il rettangolo si riferisce

Se i *bin* hanno larghezze uguali, come avviene di solito, anche **l'altezza è proporzionale alla frequenza**, e dunque ci si può riferire direttamente all'altezza

# Istogrammi: la funzione `hist`

La funzione `hist(x, bin_edges)` costruisce l'istogramma dei dati in `x` utilizzando i confini degli intervalli specificati in `bin_edges`

Per approfondimenti:

matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html

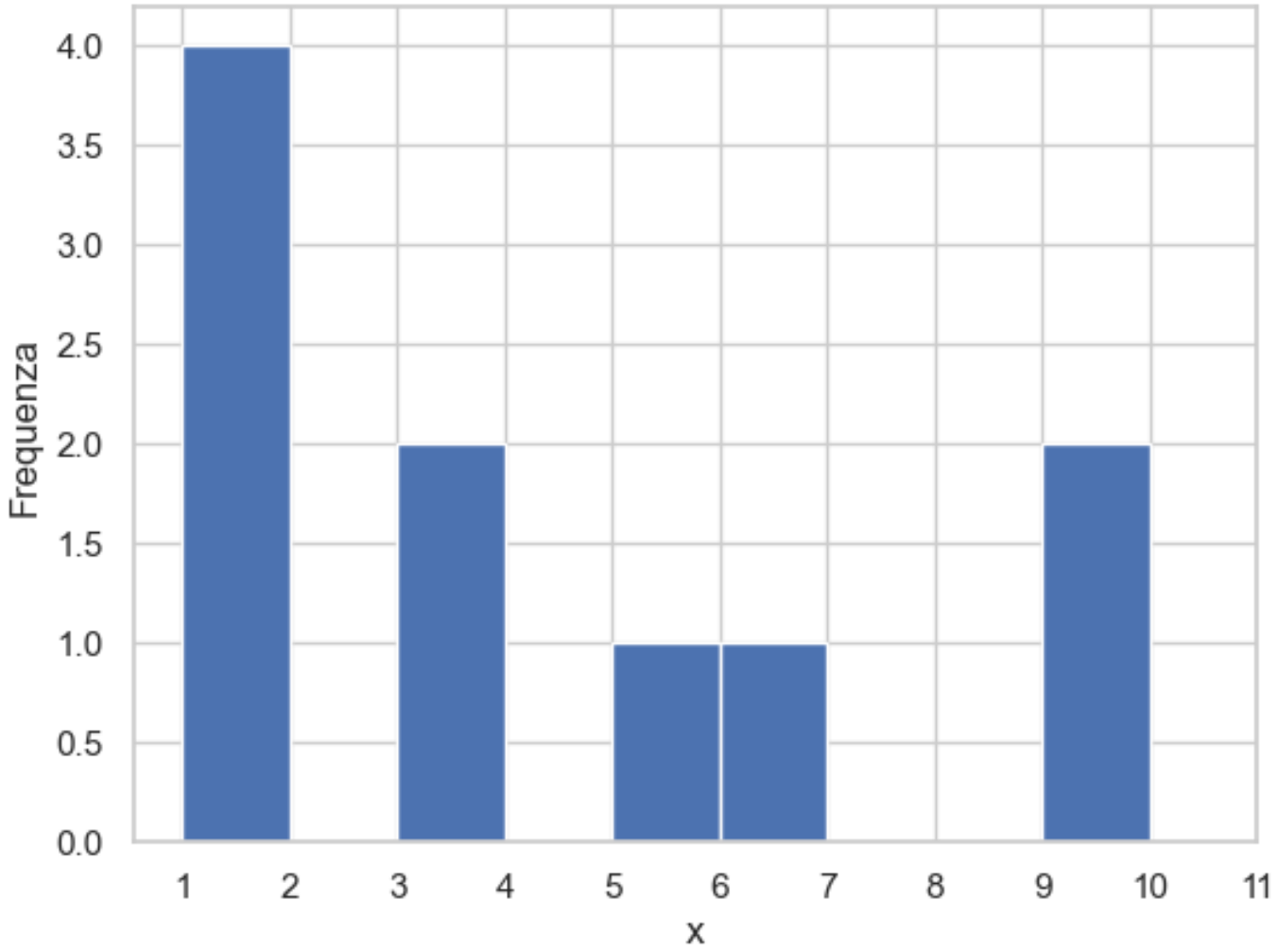
# Istogrammi: la funzione `hist`

```
import matplotlib.pyplot as plt

x = [1.1, 1.2, 1.3, 1.4, 3.2, 3.7, 5.5, 6.6, 9.5, 9.8]
bin_edges = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
#bin_edges=[1, 3, 5, 7, 9, 11]
#bin_edges=[1, 5, 10]
plt.hist(x, bin_edges)
plt.grid()
plt.xticks(range(1, 12))
plt.xlabel('x')
plt.ylabel('Frequenza')
plt.show()
```

Provate a cambiare i confini dei *bin* e i dati da rappresentare

# Istogrammi: la funzione `hist`



# Istogrammi: la funzione `hist`

Nel caso in cui i dati non siano numerici ma siano *ordinabili* (ad esempio, categorie come licenza media, diploma, laurea), non si usa un istogramma ma un **grafico a barre**, con le categorie disposte nel loro ordine e l'altezza delle barre che rappresenta il numero o la proporzione di occorrenze in ciascuna categoria

Quando i dati **non sono nemmeno ordinabili** (come nel caso di categorie non ordinate, ad esempio cibi preferiti), si utilizzano ancora i **grafici a barre**, ma con le categorie in ordine arbitrario. I grafici a barre, sebbene visivamente simili agli istogrammi, sono costruiti in modo diverso e hanno un significato diverso

# Istogrammi: la funzione `hist`

Nell'esempio precedente abbiamo indicato 'a mano' i confini degli intervalli specificati in `bin_edges`

In alternativa si può indicare come secondo argomento di `hist` il **numero** desiderato degli intervalli, lasciando che i loro confini vengano calcoalti automaticamente dalla stessa funzione, in modo tale da avere intervalli di ampiezza identica:

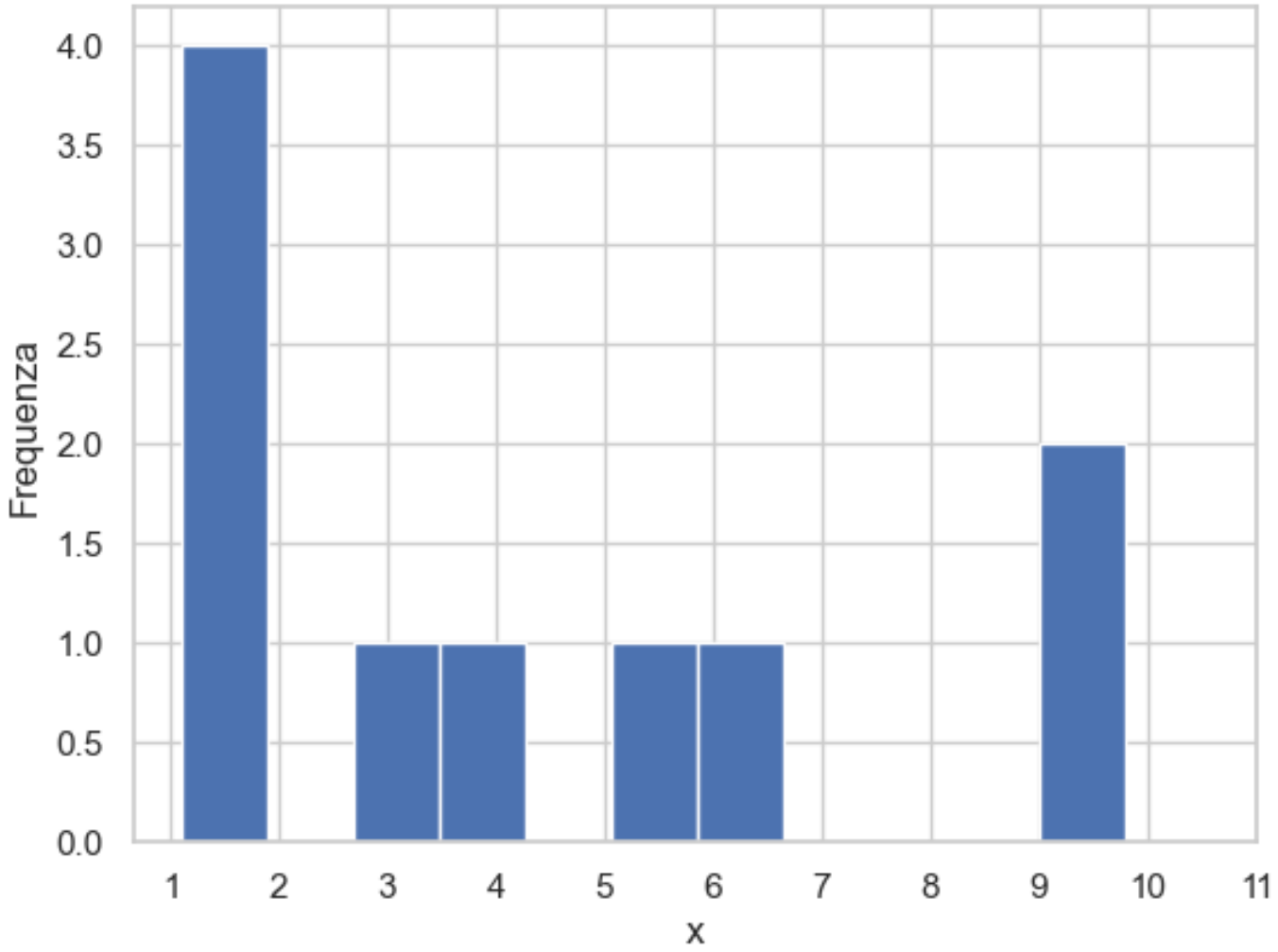
```
hist(x, n_bins)
```

# Istogrammi: la funzione `hist`

```
import matplotlib.pyplot as plt
n_bins = 11
x = [1.1, 1.2, 1.3, 1.4, 3.2, 3.7, 5.5, 6.6, 9.5, 9.8]

plt.hist(x, n_bins)
plt.grid()
plt.xticks(range(1,12))
plt.xlabel('x')
plt.ylabel('Frequenza')
plt.show()
```

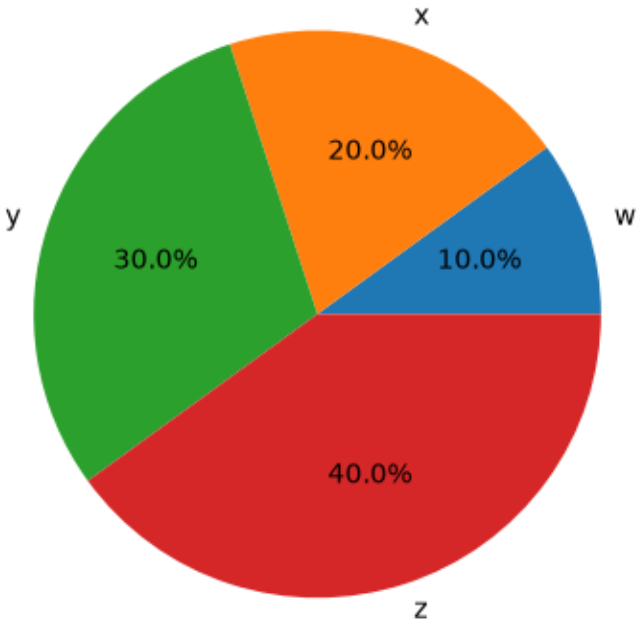
# Istogrammi: la funzione `hist`



# Grafici a torta (*pie chart*): la funzione `pie`

```
import matplotlib.pyplot as plt

valori = [10, 20, 30, 40]
etichette = ['w', 'x', 'y', 'z']
plt.pie(valori, labels = etichette, autopct = '%1.1f%%')
plt.show()
```



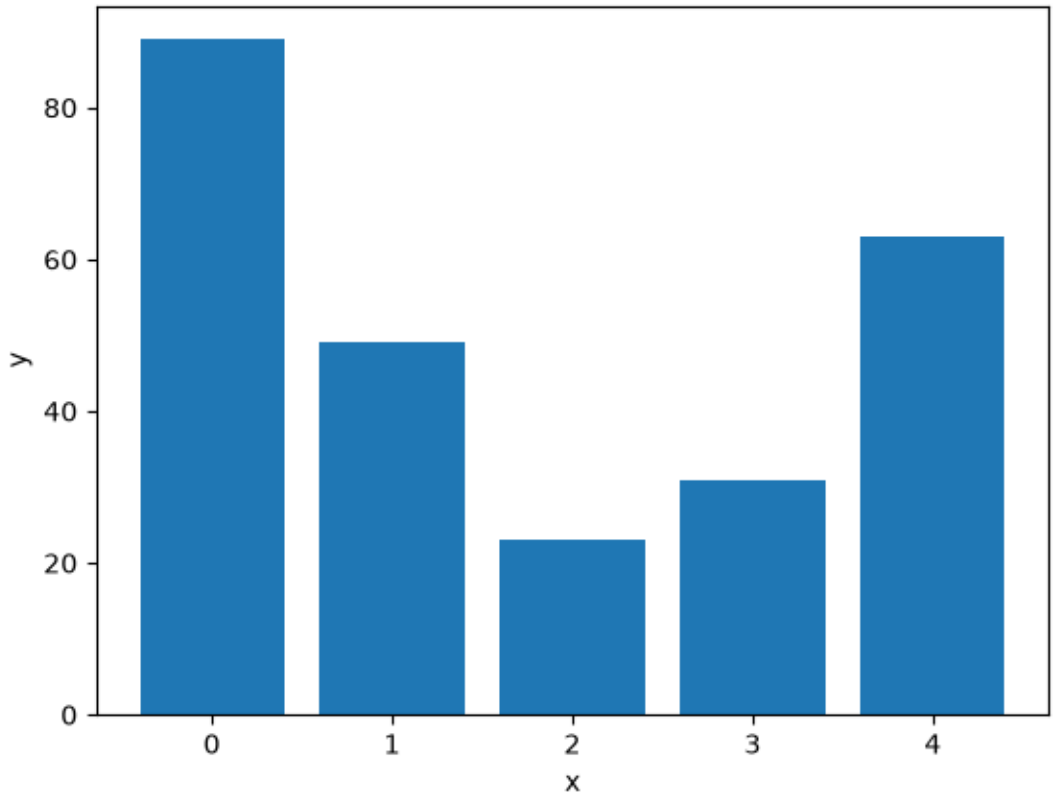
# Grafici a barre (*bar chart*): la funzione bar

```
import random
import matplotlib.pyplot as plt

x = list(range(5))  # [0, 1, 2, 3, 4]
y = [86, 47, 22, 29, 65]

plt.bar(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

# Grafici a barre (*bar chart*): la funzione bar





sAIfer Lab

Joint lab on Safety and Security of AI

# Sunto minimo

# Sunto minimo

- Calcolo di indici
  - misure di tendenza centrale (media, mediana, moda)
  - misure di dispersione (varianza, deviazione standard)
- Grafici
  - grafici X-Y: la funzione `plot()`
  - diagrammi a dispersione: la funzione `scatter()`
  - istogrammi: la funzione `hist()`
- Simulazione di eventi discreti (lancio di un dado, lancio di una moneta, estrazione di un numero della tombola)